WEEK-03

↳ 8 cells hidden

WEEK-04

↳ 8 cells hidden

WEEK-05

↳ 8 cells hidden

WEEK_6

↳ 4 cells hidden

WEEK_7

```
# Prob 1
import pandas as pd

# Create Series from list
data = [10, 20, 30, 40, 50]
series = pd.Series(data)
print("Pandas Series:")
print(series)

# Create DataFrame from dictionary
data_dict = {
    "Name": ["Ali", "Sara", "John"],
    "Age": [21, 22, 23],
    "Marks": [85, 90, 88]
}

df = pd.DataFrame(data_dict)
print("\nDataFrame from Dictionary:")
print(df)
```

```
Pandas Series:
0    10
1    20
2    30
3    40
4    50
dtype: int64

DataFrame from Dictionary:
   Name  Age  Marks
0   Ali   21     85
1  Sara   22     90
2  John   23     88
```

```
# Prob 2
import pandas as pd

# Load CSV file
df = pd.read_csv("/content/sample_data/california_housing_train.csv")

# First five rows
print("First 5 rows:")
print(df.head())

# Last five rows
print("\nLast 5 rows:")
print(df.tail())

# Dataset information
```

```python
print("\nDataset Info:")
print(df.info())
```

```
First 5 rows:
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0    -114.31     34.19                15.0       5612.0          1283.0
1    -114.47     34.40                19.0       7650.0          1901.0
2    -114.56     33.69                17.0        720.0           174.0
3    -114.57     33.64                14.0       1501.0           337.0
4    -114.57     33.57                20.0       1454.0           326.0

   population  households  median_income  median_house_value
0      1015.0       472.0         1.4936             66900.0
1      1129.0       463.0         1.8200             80100.0
2       333.0       117.0         1.6509             85700.0
3       515.0       226.0         3.1917             73400.0
4       624.0       262.0         1.9250             65500.0

Last 5 rows:
       longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
16995    -124.26     40.58                52.0       2217.0           394.0
16996    -124.27     40.69                36.0       2349.0           528.0
16997    -124.30     41.84                17.0       2677.0           531.0
16998    -124.30     41.80                19.0       2672.0           552.0
16999    -124.35     40.54                52.0       1820.0           300.0

       population  households  median_income  median_house_value
16995       907.0       369.0         2.3571            111400.0
16996      1194.0       465.0         2.5179             79000.0
16997      1244.0       456.0         3.0313            103600.0
16998      1298.0       478.0         1.9797             85800.0
16999       806.0       270.0         3.0147             94600.0

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17000 entries, 0 to 16999
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           17000 non-null  float64
 1   latitude            17000 non-null  float64
 2   housing_median_age  17000 non-null  float64
 3   total_rooms         17000 non-null  float64
 4   total_bedrooms      17000 non-null  float64
 5   population          17000 non-null  float64
 6   households          17000 non-null  float64
 7   median_income       17000 non-null  float64
 8   median_house_value  17000 non-null  float64
dtypes: float64(9)
memory usage: 1.2 MB
None
```

```python
# Prob 3
import pandas as pd

data = {
    "A": [10,20,30,40,50],
    "B": [5,15,25,35,45],
    "C": [2,4,6,8,10]
}

df = pd.DataFrame(data)

avg = df.iloc[:,1].mean()

print("Average of second column:", avg)
```

```
Average of second column: 25.0
```

```
import pandas as pd

data = {
    "A":[1,2,3,4,5,6],
    "B":[6,7,8,9,10,11],
    "C":[2,3,4,5,6,7],
    "D":[8,9,10,11,12,13]
}

df = pd.DataFrame(data)

avg = df.iloc[:5,2:4].mean()

print("Average of first 5 rows (3rd & 4th columns):")
print(avg)
```

```
Average of first 5 rows (3rd & 4th columns):
C     4.0
D    10.0
dtype: float64
```

```
import pandas as pd

data = {
    "A":[1,2,3],
    "B":[4,5,6],
    "C":[7,8,9]
}

df = pd.DataFrame(data)

row_sum = df.sum(axis=1)

print("Row wise sum:")
print(row_sum)
```

```
Row wise sum:
0    12
1    15
2    18
dtype: int64
```

```
import pandas as pd

data = {
    "A":[10,20,30],
    "B":[40,50,60],
    "C":[70,80,90]
}

df = pd.DataFrame(data)

row_avg = df.mean(axis=1)
max_avg = row_avg.max()

print("Maximum of row averages:", max_avg)
```

```
Maximum of row averages: 60.0
```

WEEK_8

```
# Problem 1
import pandas as pd

df = pd.read_csv("/content/sample_data/mnist_test.csv")

print(df)

# Check missing values
print("\nMissing values in dataset:")
print(df.isnull())

# Count missing values in each column
print("\nTotal missing values:")
print(df.isnull().sum())
```

```
      7  0  0.1  0.2  0.3  0.4  0.5  0.6  0.7  0.8  ...  0.658  0.659  0.660  \
0     2  0    0    0    0    0    0    0    0    0  ...      0      0      0
1     1  0    0    0    0    0    0    0    0    0  ...      0      0      0
2     0  0    0    0    0    0    0    0    0    0  ...      0      0      0
3     4  0    0    0    0    0    0    0    0    0  ...      0      0      0
4     1  0    0    0    0    0    0    0    0    0  ...      0      0      0
...  .. ..  ...  ...  ...  ...  ...  ...  ...  ...  ...    ...    ...    ...
9994  2  0    0    0    0    0    0    0    0    0  ...      0      0      0
9995  3  0    0    0    0    0    0    0    0    0  ...      0      0      0
9996  4  0    0    0    0    0    0    0    0    0  ...      0      0      0
9997  5  0    0    0    0    0    0    0    0    0  ...      0      0      0
9998  6  0    0    0    0    0    0    0    0    0  ...      0      0      0

      0.661  0.662  0.663  0.664  0.665  0.666  0.667
0         0      0      0      0      0      0      0
1         0      0      0      0      0      0      0
2         0      0      0      0      0      0      0
3         0      0      0      0      0      0      0
4         0      0      0      0      0      0      0
...     ...    ...    ...    ...    ...    ...    ...
9994      0      0      0      0      0      0      0
9995      0      0      0      0      0      0      0
9996      0      0      0      0      0      0      0
9997      0      0      0      0      0      0      0
9998      0      0      0      0      0      0      0

[9999 rows x 785 columns]

Missing values in dataset:
          7      0    0.1    0.2    0.3    0.4    0.5    0.6    0.7    0.8  \
0     False  False  False  False  False  False  False  False  False  False
1     False  False  False  False  False  False  False  False  False  False
2     False  False  False  False  False  False  False  False  False  False
3     False  False  False  False  False  False  False  False  False  False
4     False  False  False  False  False  False  False  False  False  False
...     ...    ...    ...    ...    ...    ...    ...    ...    ...    ...    ...
9994  False  False  False  False  False  False  False  False  False  False
9995  False  False  False  False  False  False  False  False  False  False
9996  False  False  False  False  False  False  False  False  False  False
9997  False  False  False  False  False  False  False  False  False  False
9998  False  False  False  False  False  False  False  False  False  False

      ...  0.658  0.659  0.660  0.661  0.662  0.663  0.664  0.665  0.666  \
0     ...  False  False  False  False  False  False  False  False  False
1     ...  False  False  False  False  False  False  False  False  False
2     ...  False  False  False  False  False  False  False  False  False
3     ...  False  False  False  False  False  False  False  False  False
4     ...  False  False  False  False  False  False  False  False  False
...   ...    ...    ...    ...    ...    ...    ...    ...    ...    ...
9994  ...  False  False  False  False  False  False  False  False  False
9995  ...  False  False  False  False  False  False  False  False  False
9996  ...  False  False  False  False  False  False  False  False  False
9997  ...  False  False  False  False  False  False  False  False  False
9998  ...  False  False  False  False  False  False  False  False  False

      0.667
0     False
1     False
```

```
# Problem 2
import pandas as pd

df = pd.read_csv("/content/sample_data/mnist_test.csv")

# Replace missing values with mean
df_mean = df.fillna(df.mean(numeric_only=True))

# Replace missing values with median
df_median = df.fillna(df.median(numeric_only=True))

# Drop rows with missing values
df_drop = df.dropna()

print("Filled with Mean (first 5 rows):\n", df_mean.head())
print("\nFilled with Median (first 5 rows):\n", df_median.head())
print("\nAfter Dropping Missing Values (first 5 rows):\n", df_drop.head())
```

```
Filled with Mean (first 5 rows):
    7  0  0.1  0.2  0.3  0.4  0.5  0.6  0.7  0.8  ...  0.658  0.659  0.660  \
0  2  0    0    0    0    0    0    0    0    0  ...      0      0      0
1  1  0    0    0    0    0    0    0    0    0  ...      0      0      0
2  0  0    0    0    0    0    0    0    0    0  ...      0      0      0
3  4  0    0    0    0    0    0    0    0    0  ...      0      0      0
4  1  0    0    0    0    0    0    0    0    0  ...      0      0      0
```

```
   0.661  0.662  0.663  0.664  0.665  0.666  0.667
0      0      0      0      0      0      0      0
1      0      0      0      0      0      0      0
2      0      0      0      0      0      0      0
3      0      0      0      0      0      0      0
4      0      0      0      0      0      0      0

[5 rows x 785 columns]

Filled with Median (first 5 rows):
    7  0  0.1  0.2  0.3  0.4  0.5  0.6  0.7  0.8  ...  0.658  0.659  0.660  \
0  2  0    0    0    0    0    0    0    0    0  ...      0      0      0
1  1  0    0    0    0    0    0    0    0    0  ...      0      0      0
2  0  0    0    0    0    0    0    0    0    0  ...      0      0      0
3  4  0    0    0    0    0    0    0    0    0  ...      0      0      0
4  1  0    0    0    0    0    0    0    0    0  ...      0      0      0

   0.661  0.662  0.663  0.664  0.665  0.666  0.667
0      0      0      0      0      0      0      0
1      0      0      0      0      0      0      0
2      0      0      0      0      0      0      0
3      0      0      0      0      0      0      0
4      0      0      0      0      0      0      0

[5 rows x 785 columns]

After Dropping Missing Values (first 5 rows):
    7  0  0.1  0.2  0.3  0.4  0.5  0.6  0.7  0.8  ...  0.658  0.659  0.660  \
0  2  0    0    0    0    0    0    0    0    0  ...      0      0      0
1  1  0    0    0    0    0    0    0    0    0  ...      0      0      0
2  0  0    0    0    0    0    0    0    0    0  ...      0      0      0
3  4  0    0    0    0    0    0    0    0    0  ...      0      0      0
4  1  0    0    0    0    0    0    0    0    0  ...      0      0      0

   0.661  0.662  0.663  0.664  0.665  0.666  0.667
0      0      0      0      0      0      0      0
1      0      0      0      0      0      0      0
2      0      0      0      0      0      0      0
3      0      0      0      0      0      0      0
4      0      0      0      0      0      0      0

[5 rows x 785 columns]
```

```
# Problem 3
import pandas as pd

df = pd.read_csv("/content/student_dataset_200.csv")

# Rename columns
df = df.rename(columns={
    "Student_Names": "Name",
    "Comment": "Percentage"
})

# Change data type
df["Percentage"] = df["Percentage"].astype(float)

print(df.dtypes)
print(df)
```

```
Name           object
Class           int64
Section        object
Roll No.        int64
Age             int64
Gender         object
Percentage    float64
Address        object
Contact         int64
dtype: object
                    Name  Class Section  Roll No.  Age  Gender  Percentage  \
0        Ananya Raj Khan      7       C         1   12  Female       51.26
1       Pari Gupta Patel      3       B         2   14  Female       70.45
2        Vivaan Das Iyer      8       C         3   13    Male       63.34
3       Krishna Das Nair      8       C         4   16    Male       63.78
4    Reyansh Singh Reddy      3       B         5   13  Female       77.50
..                   ...    ...     ...       ...  ...     ...         ...
195   Reyansh Prasad Das      3       A       196   17    Male       68.51
196      Anika Raj Singh      8       C       197   14  Female       57.42
197     Sai Prasad Reddy      3       B       198   17    Male       97.73
198  Reyansh Kumar Singh      1       B       199   18    Male       57.81
199      Navya Khan Iyer      3       C       200    6    Male       62.65

                                Address     Contact
0            Faridabad, Delhi - 201301  9545273051
1    Faridabad, Uttar Pradesh - 131001  9242594390
2            Faridabad, Delhi - 250001  9511493813
```

```
3     Gurugram, Uttar Pradesh - 201301  9585114439
4    New Delhi, Uttar Pradesh - 121001  9306405876
..                                ...         ...
195           Meerut, Haryana - 122001  9280934554
196             Meerut, Delhi - 131001  9738687418
197        Faridabad, Haryana - 122001  9236739351
198   Gurugram, Uttar Pradesh - 201001  9334267009
199     Rohtak, Uttar Pradesh - 121001  9649090910

[200 rows x 9 columns]
```

```
# Problem 4
import pandas as pd

df = pd.read_csv("/content/student_dataset_200.csv")

# Remove rows with missing values
df1 = df.dropna()

# Filter rows where Marks > 50
df2 = df[df["Percentage"] > 50]

# Select specific columns
df3 = df[["Name","Percentage"]]

print("Rows without missing values:\n", df1)
print("\nRows where Marks > 50:\n", df2)
print("\nSelected Columns:\n", df3)
```

```
Rows without missing values:
                     Name  Class Section  Roll No.  Age  Gender  Percentage  \
0        Ananya Raj Khan      7       C         1   12  Female       51.26
1       Pari Gupta Patel      3       B         2   14  Female       70.45
2        Vivaan Das Iyer      8       C         3   13    Male       63.34
3       Krishna Das Nair      8       C         4   16    Male       63.78
4    Reyansh Singh Reddy      3       B         5   13  Female       77.50
..                   ...    ...     ...       ...  ...     ...         ...
195   Reyansh Prasad Das      3       A       196   17    Male       68.51
196      Anika Raj Singh      8       C       197   14  Female       57.42
197     Sai Prasad Reddy      3       B       198   17    Male       97.73
198  Reyansh Kumar Singh      1       B       199   18    Male       57.81
199      Navya Khan Iyer      3       C       200    6    Male       62.65

                               Address     Contact
0            Faridabad, Delhi - 201301  9545273051
1    Faridabad, Uttar Pradesh - 131001  9242594390
2            Faridabad, Delhi - 250001  9511493813
3     Gurugram, Uttar Pradesh - 201301  9585114439
4    New Delhi, Uttar Pradesh - 121001  9306405876
..                                ...         ...
195           Meerut, Haryana - 122001  9280934554
196             Meerut, Delhi - 131001  9738687418
197        Faridabad, Haryana - 122001  9236739351
198   Gurugram, Uttar Pradesh - 201001  9334267009
199     Rohtak, Uttar Pradesh - 121001  9649090910

[200 rows x 9 columns]

Rows where Marks > 50:
                     Name  Class Section  Roll No.  Age  Gender  Percentage  \
0        Ananya Raj Khan      7       C         1   12  Female       51.26
1       Pari Gupta Patel      3       B         2   14  Female       70.45
2        Vivaan Das Iyer      8       C         3   13    Male       63.34
3       Krishna Das Nair      8       C         4   16    Male       63.78
4    Reyansh Singh Reddy      3       B         5   13  Female       77.50
..                   ...    ...     ...       ...  ...     ...         ...
195   Reyansh Prasad Das      3       A       196   17    Male       68.51
196      Anika Raj Singh      8       C       197   14  Female       57.42
197     Sai Prasad Reddy      3       B       198   17    Male       97.73
198  Reyansh Kumar Singh      1       B       199   18    Male       57.81
199      Navya Khan Iyer      3       C       200    6    Male       62.65

                               Address     Contact
0            Faridabad, Delhi - 201301  9545273051
1    Faridabad, Uttar Pradesh - 131001  9242594390
2            Faridabad, Delhi - 250001  9511493813
3     Gurugram, Uttar Pradesh - 201301  9585114439
4    New Delhi, Uttar Pradesh - 121001  9306405876
..                                ...         ...
195           Meerut, Haryana - 122001  9280934554
196             Meerut, Delhi - 131001  9738687418
197        Faridabad, Haryana - 122001  9236739351
198   Gurugram, Uttar Pradesh - 201001  9334267009
199     Rohtak, Uttar Pradesh - 121001  9649090910

[200 rows x 9 columns]
```

WEEK_9

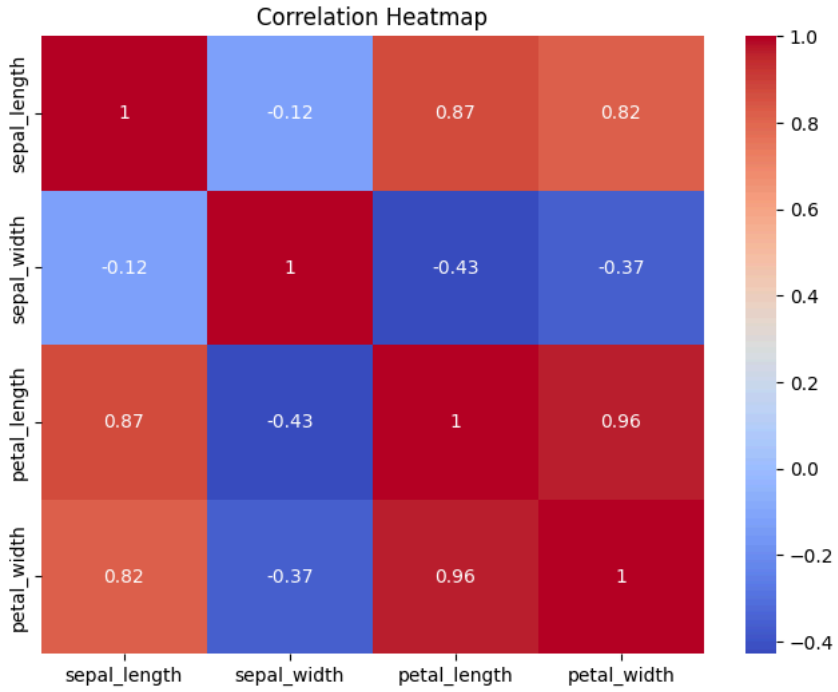
```
# Problem 1
import pandas as pd
import seaborn as sns


df = sns.load_dataset('iris')

print("First 5 rows:")
print(df.head())


print("\nDataset Info:")
print(df.info())

print("\nSummary Statistics (Numerical Columns):")
print(df.describe())
```

```
First 5 rows:
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None

Summary Statistics (Numerical Columns):
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.057333      3.758000     1.199333
std        0.828066     0.435866      1.765298     0.762238
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000
```

```
#Problem 2

import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt

df = sns.load_dataset('iris')

num_cols = df.select_dtypes(include=np.number).columns

corr_matrix = df[num_cols].corr()

print("Correlation Matrix:")
print(corr_matrix)

plt.figure(figsize=(8,6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

threshold = 0.9

upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
```

```
to_drop = [column for column in upper.columns if any(upper[column] > threshold)]

print("\nHighly correlated columns to drop:", to_drop)

df_reduced = df.drop(columns=to_drop)

print("\nRemaining columns:")
print(df_reduced.columns)
```

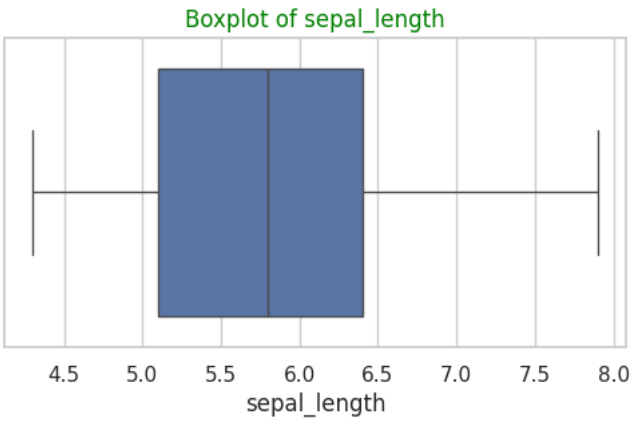
```
Correlation Matrix:
              sepal_length  sepal_width  petal_length  petal_width
sepal_length      1.000000    -0.117570      0.871754     0.817941
sepal_width      -0.117570     1.000000     -0.428440    -0.366126
petal_length      0.871754    -0.428440      1.000000     0.962865
petal_width       0.817941    -0.366126      0.962865     1.000000
```

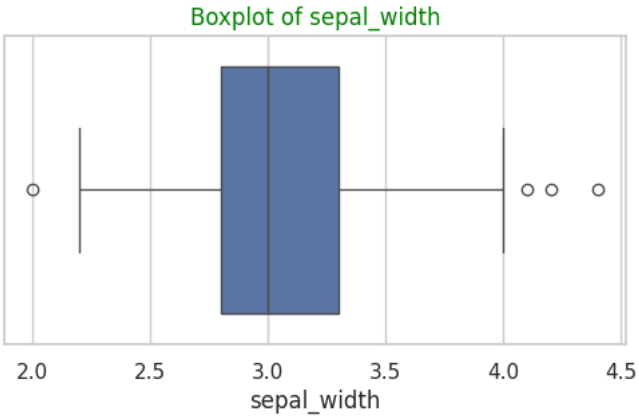


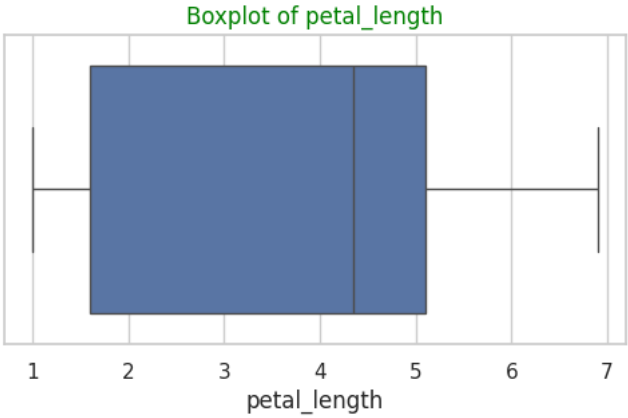

```
Highly correlated columns to drop: ['petal_width']

Remaining columns:
Index(['sepal_length', 'sepal_width', 'petal_length', 'species'], dtype='object')
```

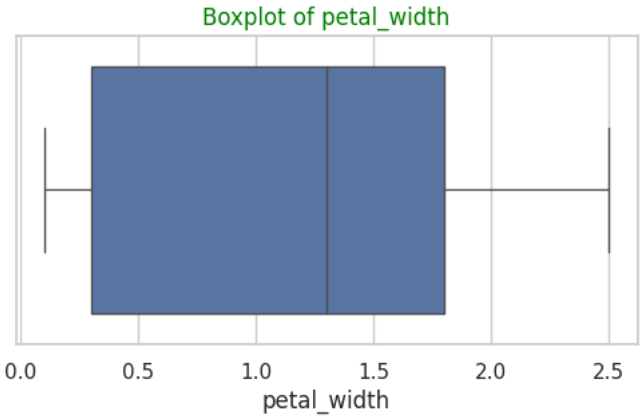
```
# Problem 3
import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt

df = sns.load_dataset('iris')



num_cols = df.select_dtypes(include=np.number).columns


for col in num_cols:
    plt.figure(figsize=(6,3))
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot of {col}", color="green")
    plt.show()

def remove_outliers_iqr(data, columns):
    df_clean = data.copy()

    for col in columns:
        Q1 = df_clean[col].quantile(0.25)
        Q3 = df_clean[col].quantile(0.75)
        IQR = Q3 - Q1

        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR

        df_clean = df_clean[(df_clean[col] >= lower) & (df_clean[col] <= upper)]
```

```
    return df_clean

df_clean = remove_outliers_iqr(df, num_cols)

print("Original shape:", df.shape)
print("After removing outliers:", df_clean.shape)
```







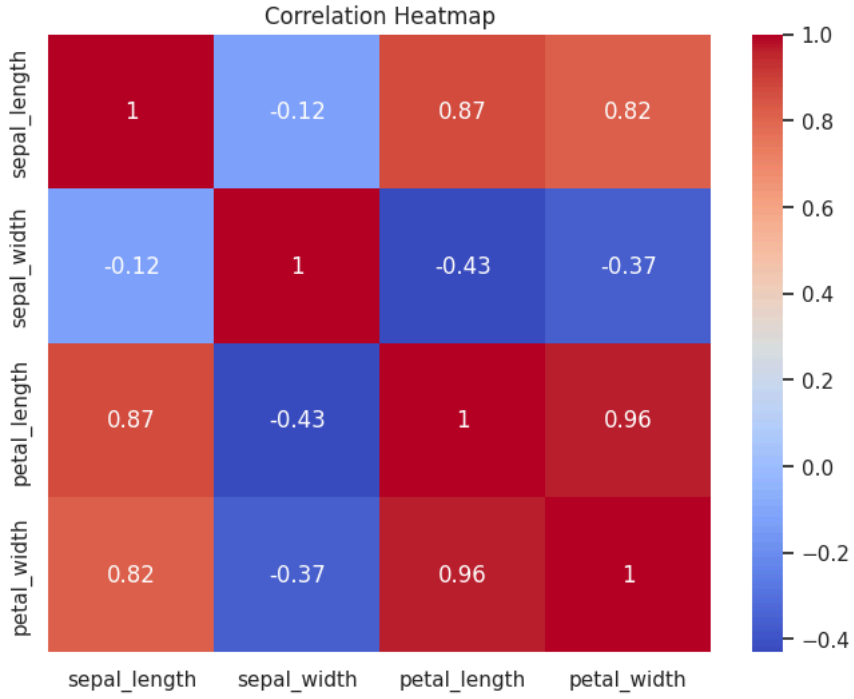


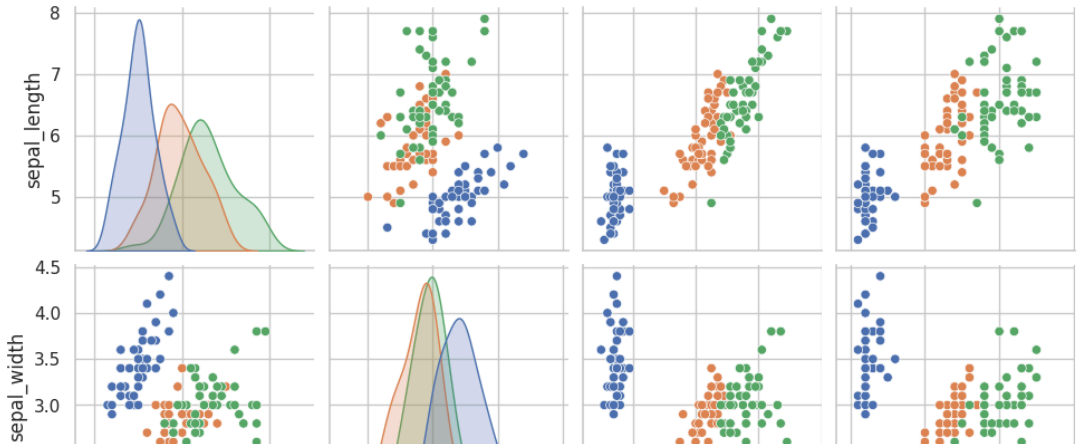

```
Original shape: (150, 5)
After removing outliers: (146, 5)
```

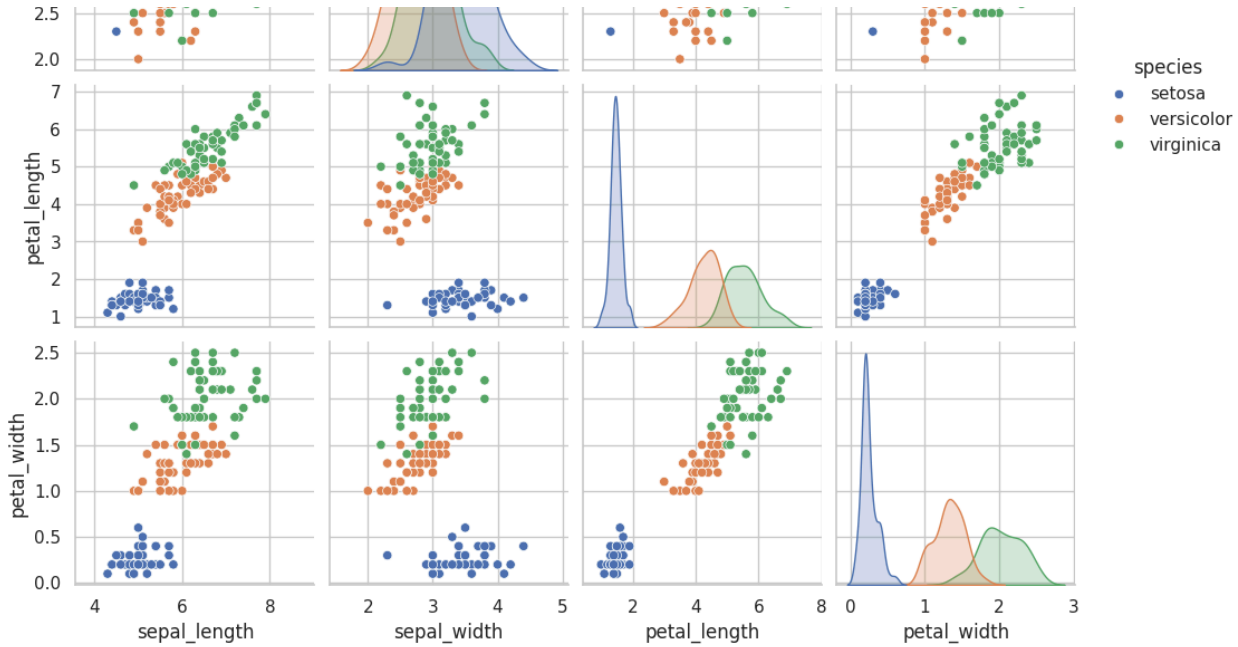
```
# Problem 4
import pandas as pd
```

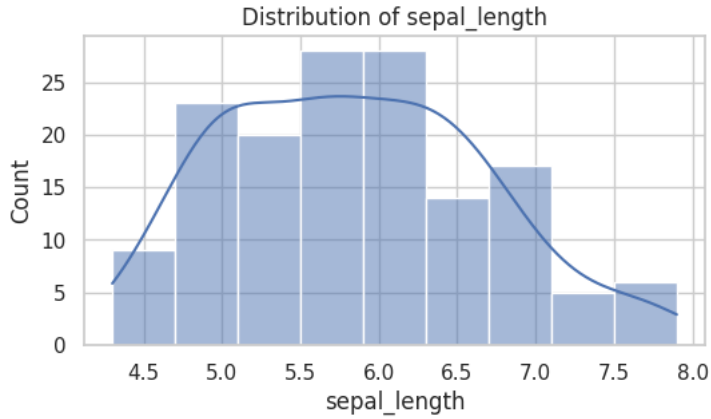
```python
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt

sns.set(style="whitegrid")


df = sns.load_dataset('iris')


print("First 5 rows:")
print(df.head())

print("\nDataset Info:")
df.info()

print("\nSummary Statistics:")
print(df.describe())



print("\nMissing Values:")
print(df.isnull().sum())


num_cols = df.select_dtypes(include=np.number).columns

corr = df[num_cols].corr()

plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

sns.pairplot(df, hue='species')
plt.show()


for col in num_cols:
    plt.figure(figsize=(6,3))
    sns.histplot(df[col], kde=True)
    plt.title(f"Distribution of {col}")
    plt.show()

for col in num_cols:
    plt.figure(figsize=(6,3))
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot of {col}")
    plt.show()

def remove_outliers_iqr(data, columns):
    df_clean = data.copy()

    for col in columns:
        Q1 = df_clean[col].quantile(0.25)
        Q3 = df_clean[col].quantile(0.75)
        IQR = Q3 - Q1

        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR

        df_clean = df_clean[(df_clean[col] >= lower) & (df_clean[col] <= upper)]

    return df_clean

df_clean = remove_outliers_iqr(df, num_cols)

print("\nShape before:", df.shape)
print("Shape after outlier removal:", df_clean.shape)

print("\nCleaned Data:")
print(df_clean.head())
```

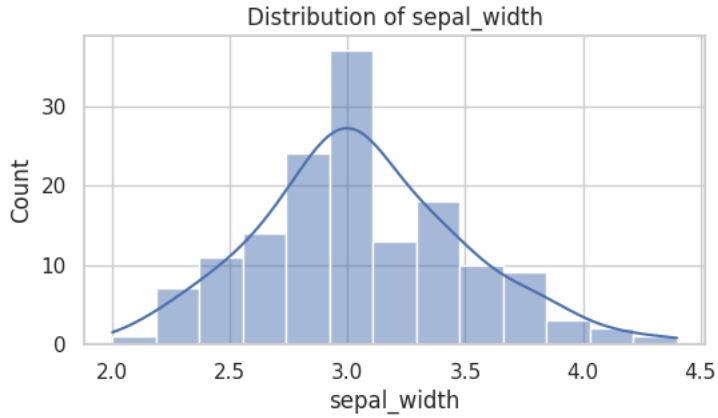
```
First 5 rows:
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB

Summary Statistics:
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.057333      3.758000     1.199333
std        0.828066     0.435866      1.765298     0.762238
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000

Missing Values:
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

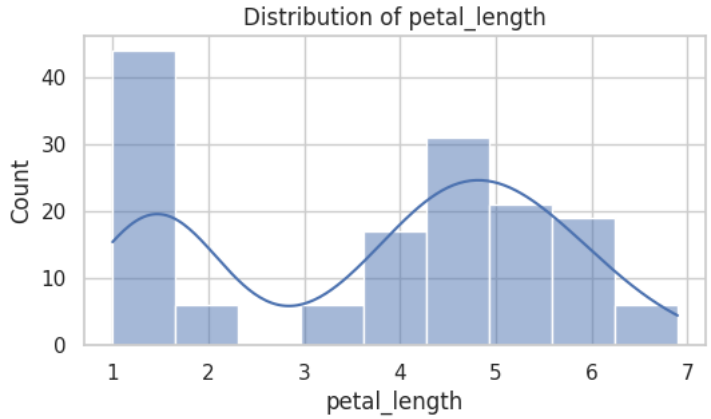


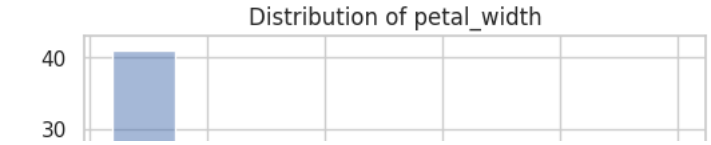


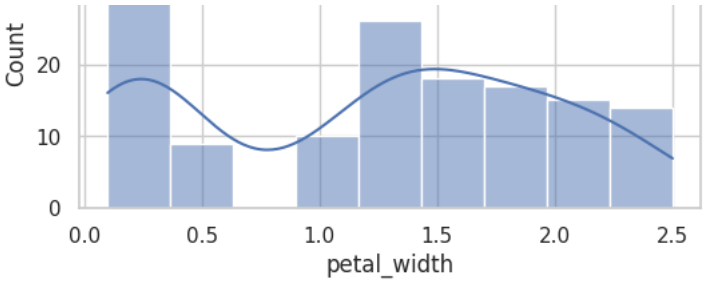

species
setosa
versicolor
virginica
sepal_length
sepal_width
petal_length
petal_width

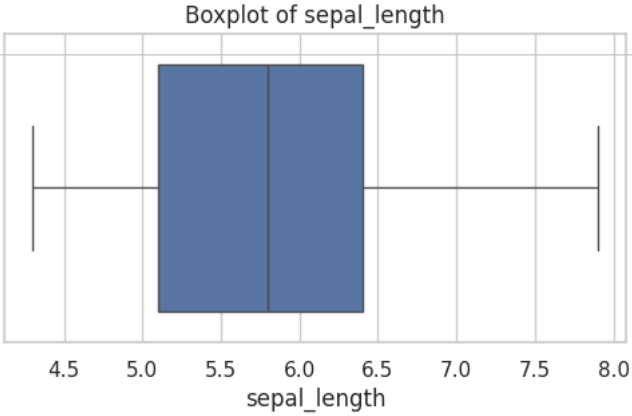

Distribution of sepal_length
Count
sepal_length

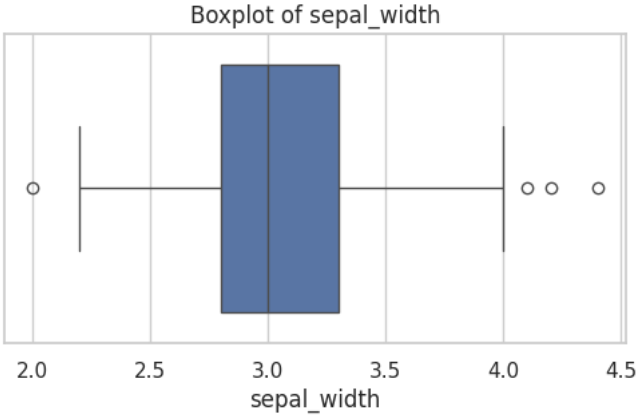

Distribution of sepal_width
Count
sepal_width

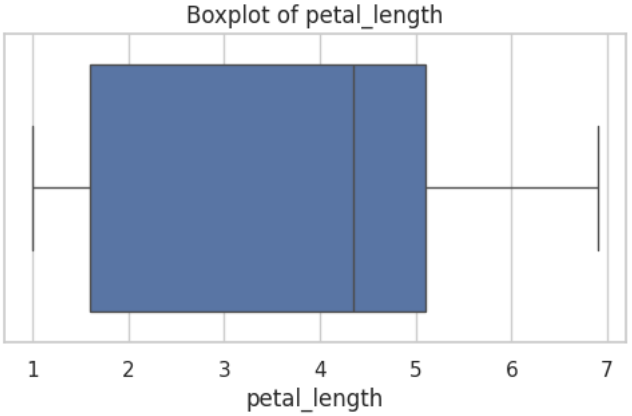

Distribution of petal_length
Count
petal_length

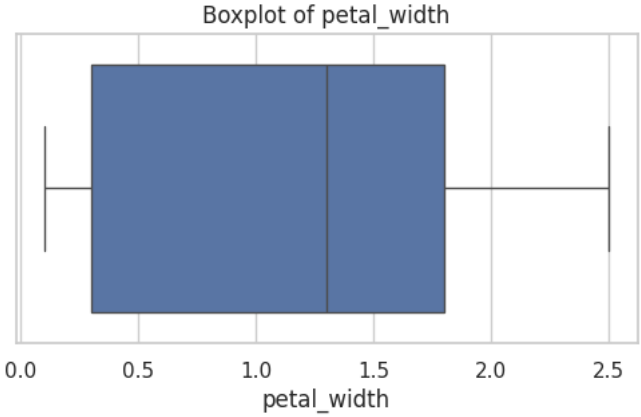

Distribution of petal_width












```
Shape before: (150, 5)
Shape after outlier removal: (146, 5)
```

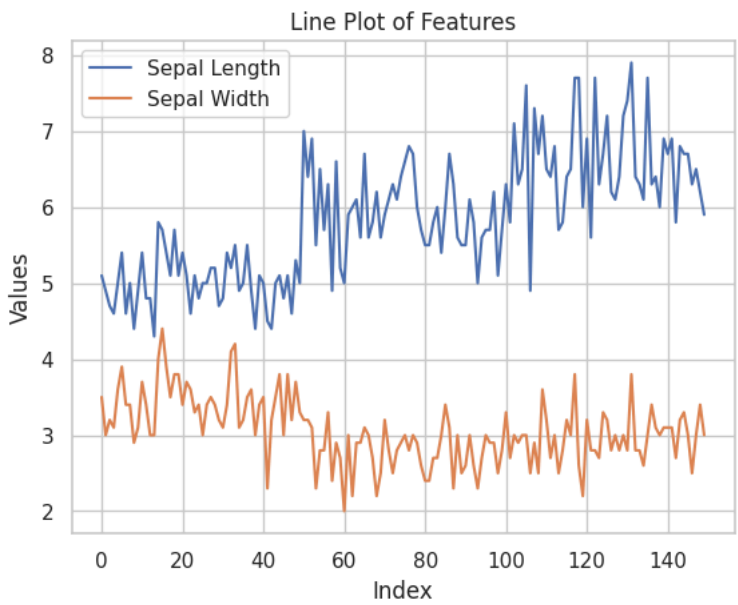
WEEK_10

```
Cleaned Data:
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
```

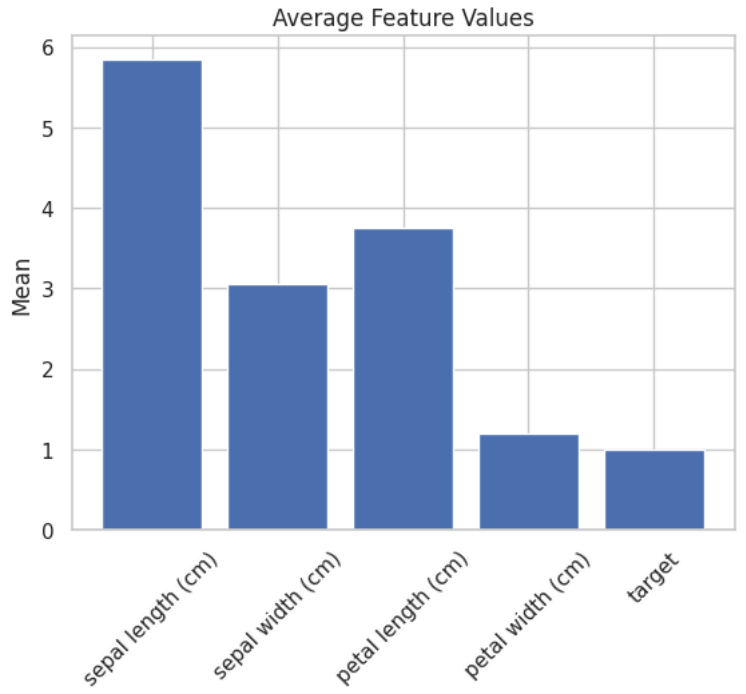
```
# Problem 1

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris


iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

print(df.head())
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target
0       0
1       0
2       0
3       0
4       0
```

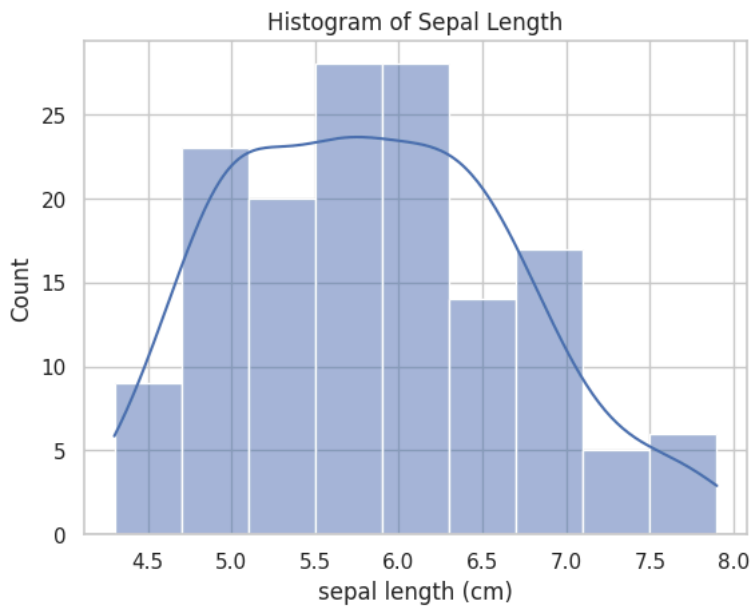
```
plt.figure()
plt.plot(df['sepal length (cm)'], label='Sepal Length')
plt.plot(df['sepal width (cm)'], label='Sepal Width')

plt.title("Line Plot of Features")
plt.xlabel("Index")
plt.ylabel("Values")
plt.legend()
plt.show()
```



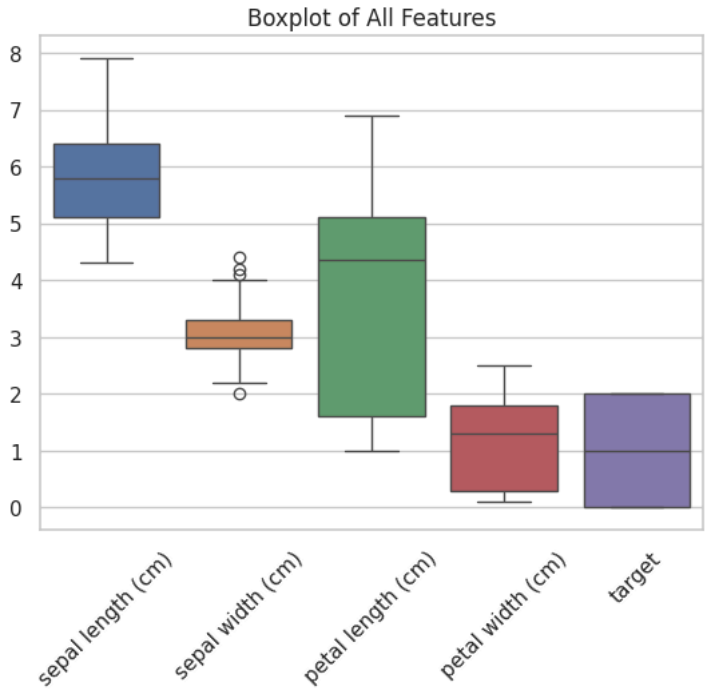

```
mean_values = df.mean()

plt.figure()
plt.bar(mean_values.index, mean_values.values)

plt.title("Average Feature Values")
plt.xticks(rotation=45)
plt.ylabel("Mean")
plt.show()
```



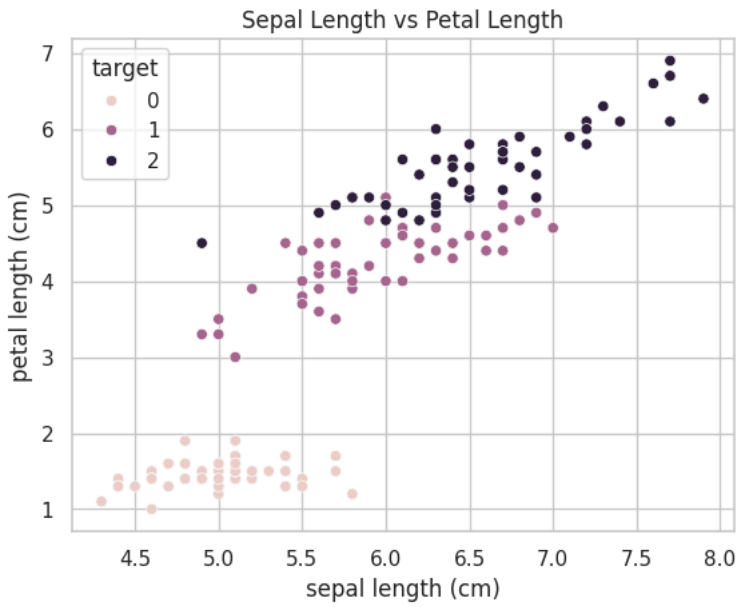

```
# Problem 2
import seaborn as sns

plt.figure()
sns.histplot(df['sepal length (cm)'], kde=True)

plt.title("Histogram of Sepal Length")
plt.show()
```



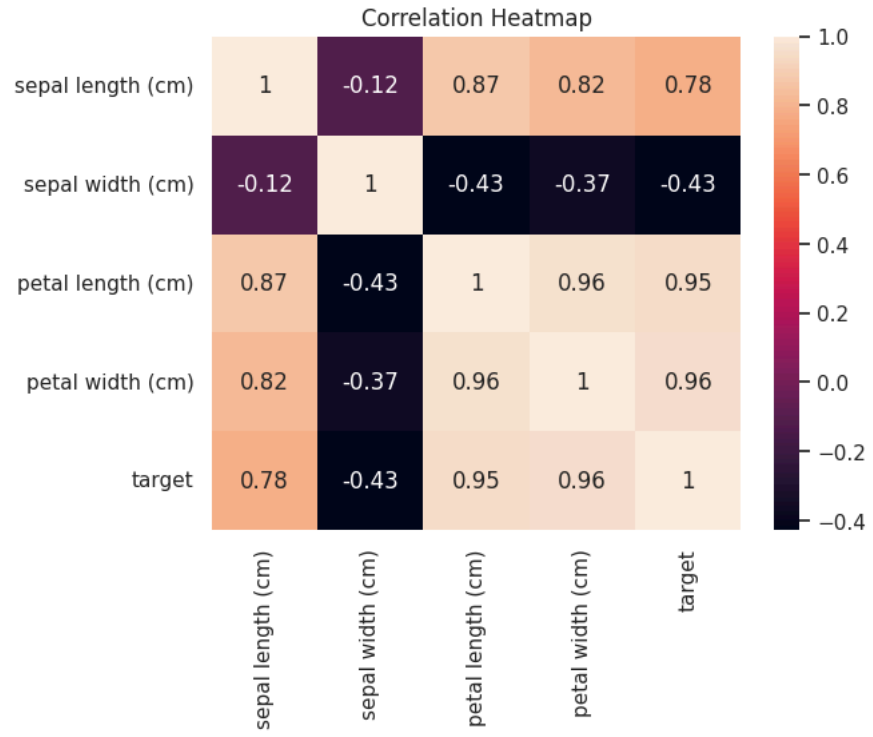

```
plt.figure()
sns.boxplot(data=df)

plt.title("Boxplot of All Features")
plt.xticks(rotation=45)
plt.show()
```




```
# Problem 3
plt.figure()
sns.scatterplot(x='sepal length (cm)', y='petal length (cm)', hue='target', data=df)

plt.title("Sepal Length vs Petal Length")
plt.show()
```




```
plt.figure()
corr = df.corr()

sns.heatmap(corr, annot=True)

plt.title("Correlation Heatmap")
plt.show()
```



WEEK_11

```
# Problem 1
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split


iris = load_iris()

data = pd.DataFrame(iris.data, columns=iris.feature_names)
data['target'] = iris.target


print("Dataset preview:")
print(data.head())


X = data.drop('target', axis=1)
y = data['target']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

#Output shapes
print("\nTraining data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
Dataset preview:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target
0       0
1       0
2       0
3       0
4       0
```

```
Training data shape: (120, 4)
Testing data shape: (30, 4)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

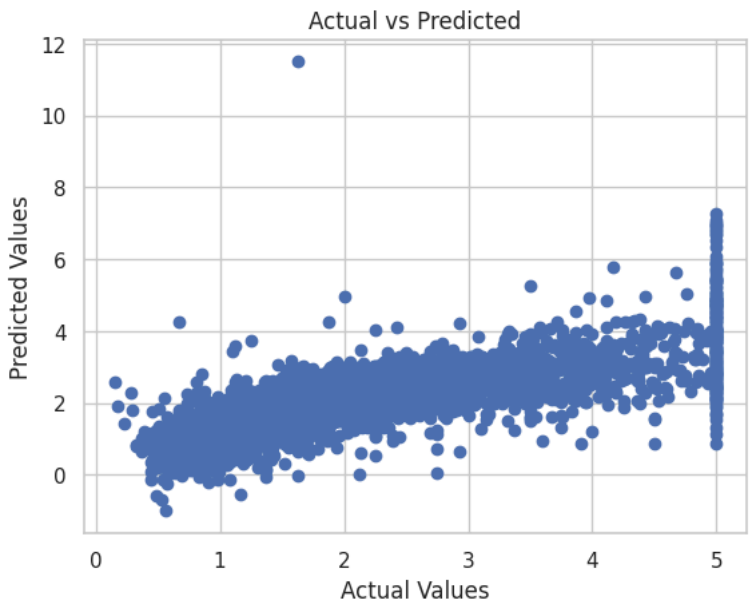
```
Accuracy: 1.0
```

```
# Problem 2

import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

housing = fetch_california_housing()


data = pd.DataFrame(housing.data, columns=housing.feature_names)
data['target'] = housing.target

print("Dataset Preview:")
print(data.head())


X = data.drop('target', axis=1)
y = data['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)


model = LinearRegression()
model.fit(X_train, y_train)


y_pred = model.predict(X_test)


mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nModel Evaluation:")
print("Mean Squared Error:", mse)
print("R2 Score:", r2)

print("\nModel Coefficients:")
for name, coef in zip(X.columns, model.coef_):
    print(f"{name}: {coef}")
```

```
Dataset Preview:
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

   Longitude  target
0    -122.23   4.526
1    -122.22   3.585
2    -122.24   3.521
3    -122.25   3.413
4    -122.25   3.422

Model Evaluation:
Mean Squared Error: 0.5558915986952444
R2 Score: 0.5757877060324508
```

```
Model Coefficients:
MedInc: 0.44867490966571666
HouseAge: 0.009724257517904806
AveRooms: -0.12332334282795815
AveBedrms: 0.7831449067929722
Population: -2.029620580100075e-06
AveOccup: -0.003526318487134238
Latitude: -0.4197924865883604
Longitude: -0.433708064963987
```

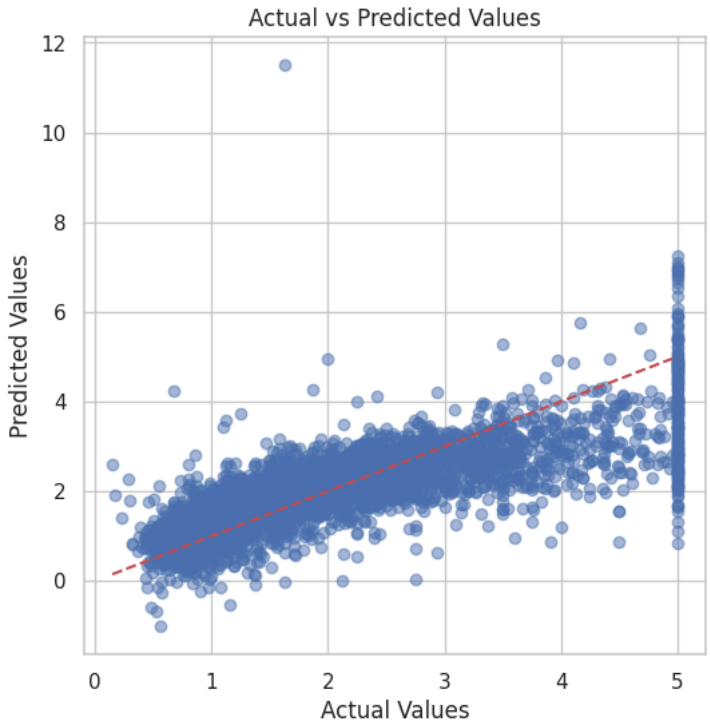
```
import matplotlib.pyplot as plt

plt.scatter(y_test, y_pred)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Actual vs Predicted")
plt.show()
```




```
# Problem 3

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error


data = fetch_california_housing()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target


X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("Model Performance:")
print("MAE (Mean Absolute Error):", mae)
print("RMSE (Root Mean Squared Error):", rmse)
```

```
Model Performance:
MAE (Mean Absolute Error): 0.5332001304956553
RMSE (Root Mean Squared Error): 0.7455813830127764
```

```
plt.figure(figsize=(6,6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Actual vs Predicted Values")

plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')

plt.show()
```




```
# Problem 4

import pandas as pd
import numpy as np

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

data = fetch_california_housing()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model 1: Linear Regression

lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
lr_pred = lr_model.predict(X_test)

lr_mae = mean_absolute_error(y_test, lr_pred)
lr_rmse = np.sqrt(mean_squared_error(y_test, lr_pred))

# Model 2: Decision Tree Regression
```